

Introduction to LLVM with Compiler basics

Compiler Engineering #3

Yuri Takigawa

2026/07/11

M1, Taura Laboratory, Creative Information, Graduate School of Information Science Technology, The University of Tokyo

Contents

1. Session Bridge 2

2. Pass 7

3. Implementation of a pass 13

1. Session Bridge

Where We Are

By using a general **IR**, your compiler can use the same optimizations and infra for a large variety of languages, targets, and OSs; *Reusability*.

Previous session:

- Structure of IR Object (Module, Function, Basic Block, Instruction)
- Key concept about CFG (backedge, critical edge, irreducible CFG)
- LLVM IR syntax (type, `getelementptr` instr, target triple & data layout)
- format for optimization; SSA (phi instruction for explicit CFG handling, dominance and def-use relationship)

Today:

Optimization overview

A lot of work of a backend compiler engineer is about improving the performance of generated code; write *optimizations*.

The ultimate goal of the optimization is

*to balance the **legality** and the **profitability***

Legality is a concept about whether the optimization preserve the semantics of the program.

- No Unsigned Wrap (NUW) / No Signed Wrap (NSW) for the behavior hint on integer overflow/underflow.
- Fast-math flags (FMF) hints on specific instructions.

More generally, optimizations must consider their *side effects*.

Example: In the following sequence, is it legal to replace `val3` directly with `val1`?

```
val1 = A[0];  
B[1] = val2;  
val3 = A[0];
```

- The answer depends on which memory location `B[1]` points to.
- If `B[1]` and `A[0]` alias, this is not legal.

No silver bullet to decide whether something is profitable, it depends on what you are trying to achieve. It is important to understand both *pros* and *cons* of the optimization.

For the example of **inlining optimization**, replacing calls to functions with the content of their body

- removes the overhead of executing the call, and exposes more optimization opportunities by breaking the boundaries of the call, and so on.; **pros**
- may increase the code size of your final executable and thrash your instruction cache, or end up oversubscribing the physical registers, and so on.; **cons**

See *Table 4.2* for the example.

2. Pass



What is a pass?

A **pass** is a class that does the followings for the specifiable *scope*.

- Encapsulates a **transformation** (e.g., an analysis, an optimization).
- Describes the dependencies of this transformation.
 - E.g., transformation A needs to have access to the dominator tree analysis B
- Returns the *effect* of this transformation on the IR (e.g., modify the CFG)

The available scopes are

- **Module**: The full IR of the current module.
- **CGSCC**: Call-graph strongly connected component is a strongly connected subset of the functions¹ within a module.
- **Function**: The IR representing a single function
- **Loop**: The IR representing a single loop

¹強連結成分 of a directed graph, where the nodes are the functions, and the edges are the possible function calls from the caller and the callee. “Strongly connected” means that each function within this subset can reach all the other ones in this subset by traversing the call graph.

What is a pass manager?

A **pass manager** is a driver for the set of passes that you want to run.

It fulfills three major functionalities:

- It provides a structure to run passes in a specific order.
- It makes sure that the dependencies of a pass are properly executed before the pass itself.
- It decides whether it preserves or invalidates the various analyses base on the passes' *effects*.

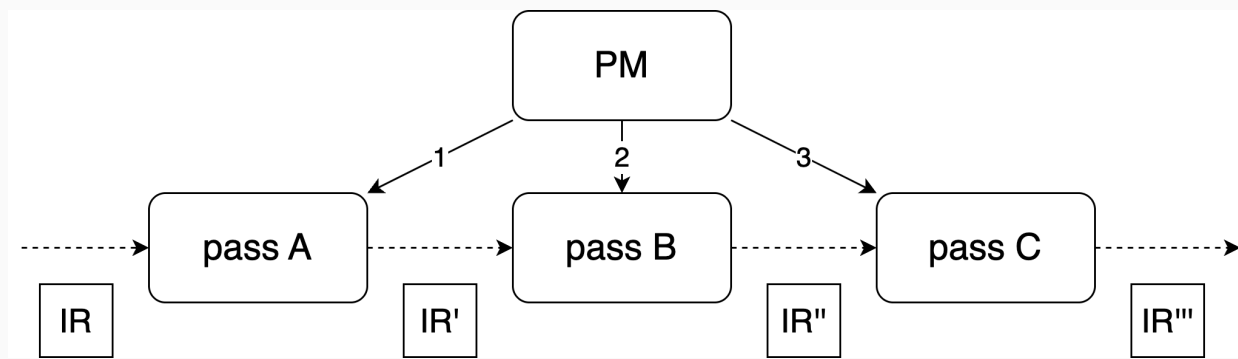


Figure 1: Structure of passes, pass manager (PM), IR

The guarantees on the order in which the IR is visited

When a pass manager invokes a pass, it also provides specific guarantees on the order in which the IR is visited.

The policy is simple: a children-to-parents traversal.

- For loop-scoped passes, loops are visited from innermost one to outermost ones.
- For CGSCC-scoped passes, a pass manager invokes the pass on the leaf *SCC* regions first then moves up in the call graph.

Inner workings of pass managers

This will help you

- understand why certain analysis passes need to be (re)run
 - fix compile-time issues by making an informed decision on whether you should preserve some analysis or adjust your pass pipeline.
1. Before running a pass, they check if the analyses that this pass relies on are available:
 - a. If yes, nothing needs to be done.
 - b. If no, they run these analyses beforehand.
 2. They run the pass.
 3. If the pass modified the IR, they check what kind of information the pass affects and not preserved.

The legacy and new pass manager

The LLVM infrastructure is in the middle of a transition from the previous pass manager to the new one.

3. Implementation of a pass

The implementation template differs between **legacy pass manager** and new **pass manager**, but they have common information that you must provide

- What is the scope of a pass?
- What analyses does this pass depend on?
- What effect does this pass have on the input IR?

Legacy pass manager: Using the proper base class

Creating a pass for a specific scope implies *inheriting from the related scoped pass class nad implementing the `runOnXXX`¹ method.*

Scope	Inherit from	Method to override
Module	ModulePass	runOnModule(Module&)
CGSCC	CallGraphSCCPass	runOnSCC(CallGraphSCC&)
Function	FunctionPass	runOnFunction(Function&)
Loop	LoopPass	runOnLoop
Region	RegionPass	runOnRegion(Region*, GPassManager&)
MachineFunction	MachineFunctionPass	runOnMachineFunction(MachineFunction&)

The skeleton of your pass looks like:

```
class MyPass * public XXXPass {
public:
    bool runOnXXX(/*Proper IR arguments*/) override;
};
```

¹The Boolean returned by `runOnXXX` method tells the pass manager whether the related pass made any changes to the IR fed to this method.

Legacy pass manager: overview

Users define a top-level pipeline using Pass Manager as shown below. In the `PM.run()`,

1. PM calls `getAnalysisUsage(AU)` for every registered pass.
2. PM collects dependencies from `AU.addRequired` called in `getAnalysisUsage()`
3. PM run all the required Analysis Passes in advance.
4. PM calls `runOnXXX(s)` following the specific order.

```
// instantiate PM
legacy::PassManager PM;

// register pass to PM
PM.add(new MyPass());

PM.run(Module);
```

```
struct MyPass : public FunctionPass {
    static char ID;
    MyPass() : FunctionPass(ID) {}

    void getAnalysisUsage(AnalysisUsage &AU) const override {
        AU.addRequired<DominatorTreeWrapperPass>();
        AU.setPreservesCFG();
    }

    bool runOnFunction(Function &F) override {
        auto &DT = getAnalysis<DominatorTreeWrapperPass>().getDomTree();
    }
};
```

Legacy pass manager: Expressing the dependencies of a pass

To describe the dependencies of a pass (and of an analysis), you must describe two elements

Which analyses the pass depends on

All you need to do is to *override the* `Pass::getAnalysisUsage(AnalysisUsage &AU)` *method*, which will be called by pass manager at the beginning of `PM.run()` for pass scheduling.

- You can add dependencies to provided `AnalysisUsage` object by calling `AnalysisUsage::addRequired</*PassClass*>()` for each analysis.
- You will be able to use the result of the analysis in the `runOnXXX` by calling `Pass::getAnalysis</*PassClass*>()`

How this pass and its analyses are initialized

- You need to provide a specific ID to register your pass by adding a `static char ID` field to your class and initializing this field to any value outside of the class (like `char MyPass::ID = 0;`)

Legacy pass manager: preserving analyses

Make sure the boolean status returned by `runOnXXX` accurately captures whether you made changes to the IR.

The implication of this boolean is:

If a pass returns `true`, PM assumes that each analysis previously performed is invalidated, meaning that if a later pass needs one of them. Thus, PM will have to reschedule them to recompute them.

Sorry, I was running out of time. I will upload next week (off on schedule)...

LLVM IR API corresponding to each component

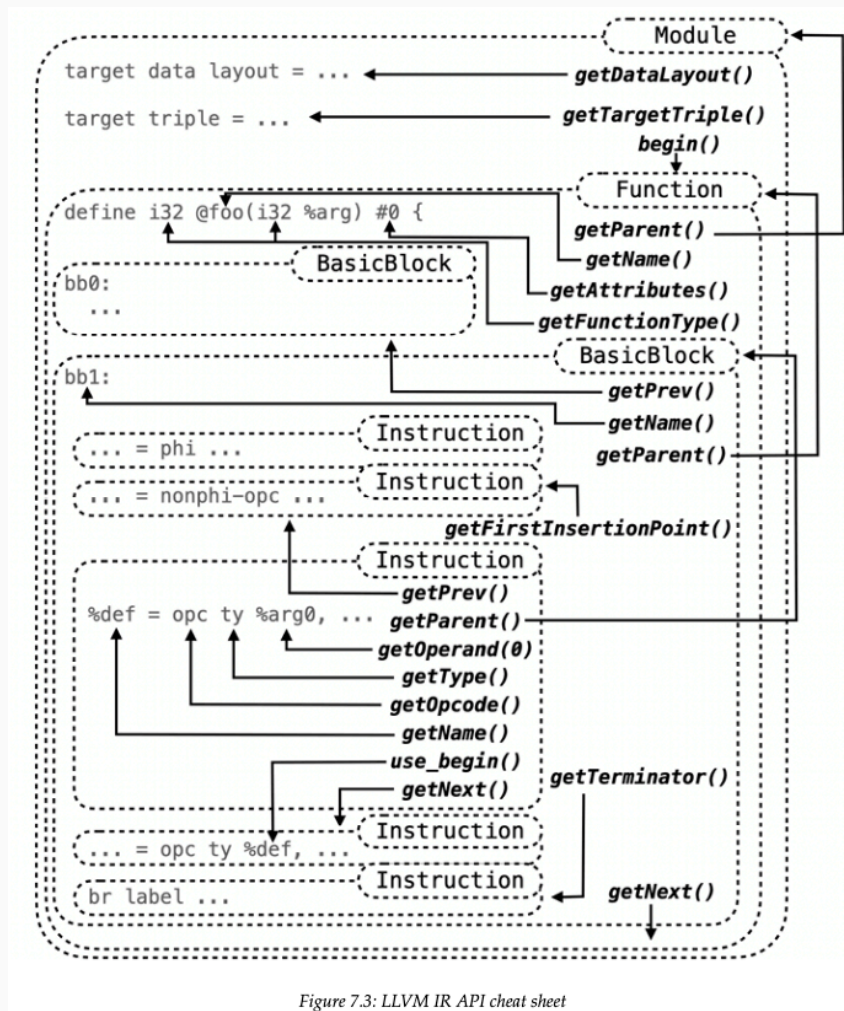


Figure 7.3: LLVM IR API cheat sheet

Every component at every level has corresponding API to read/write IR.

Implementation Tasks

Work on **Your turn** of Chapter 5.

- Writing your own pass, one for legacy pass manager, and one for the new pass manager.
- Writing your own pass pipeline.